# ICPC

Problema J

## Problem J. Judge Meetings

| | |
|---|---|
| Source file name: | Judge.c, Judge.cpp, Judge.java, Judge.py |
| Input: | Standard |
| Output: | Standard |

The judges for the UCF Local Programming Contest have very busy summers. In order to have effective meetings, at least 3 judges have to be present. Since you enjoy Locals and want the contest to go on, given their vacation schedules, help the judges figure out how many days over the summer they could have meetings.

Given the list of days that each judge is away on vacation, determine the number of days that at least 3 judges could meet (are not away on vacation).

### Input

The first input line contains two integers: $n$ ($3 \le n \le 10$), indicating the number of judges, and $m$ ($30 \le m \le 120$), indicating the number of days for summer vacation. These days are numbered from 1 to $m$. Each of the next $n$ input lines will contain information about each judge's vacations. The first item on each of these input lines is an integer, $v$ ($1 \le v \le 10$), indicating the number of vacations that judge has taken. This will be followed by $v$ pairs of integers, $s$ and $e$, indicating that the judge took a vacation starting on day $s$ of the summer and ending on day $e$ of the summer. For all pairs of $s$ and $e$, $1 \le s \le e \le m$. For each judge, no vacations will overlap but it's possible that a judge could end a vacation on one day and leave for another vacation on the following day, meaning that judge is not available for a meeting at all between those two vacations. The vacations for a judge are not necessarily listed in order, i.e., the vacations for a judge are listed in a random order.

### Output

Print the number of days during the summer that at least 3 judges are available to meet (not on vacation).

### Example

| Input | Output |
|---|---|
| 5 90<br>2 30 40 50 60<br>1 1 45<br>1 40 90<br>2 33 40 41 50<br>1 45 90 | 62 |
| 3 60<br>2 30 39 1 10<br>1 1 20<br>1 45 60 | 14 |

```
#include <bits/stdc++.h>
using namespace std;
```

```
int main() {
    int n, m;
    cin >> n >> m;

    vector<int> dias(m + 1, 0);

    for (int i = 0; i < n; i++) {
        int v;
        cin >> v;  // número de vacaciones del juez
        for (int j = 0; j < v; j++) {
            int inicio, fin;
            cin >> inicio >> fin;
            for (int d = inicio; d <= fin; d++) {
                dias[d]++;  // ese día el juez está fuera
            }
        }
    }

    int dias_reunion = 0;
    for (int d = 1; d <= m; d++) {
        int disponibles = n - dias[d];
        if (disponibles >= 3) dias_reunion++;
    }

    cout << dias_reunion << endl;
}
```

Problema D

# Problem D. Dart Game

| | |
|---|---|
| Source file name: | Dart.c, Dart.cpp, Dart.java, Dart.py |
| Input: | `Standard` |
| Output: | `Standard` |

Consider an $n \times n$ grid where $n$ is an odd number between 3 and 21, inclusive. Label the cells in the grid as follows:

| | | | | |
|---|---|---|---|---|
| (1,1) | (1,2) | (1,3) | … | (1,21) |
| (2,1) | (2,2) | (2,3) | … | |
| (3,1) | … | | | |
| … | | | | |
| (21,1) | (21,2) | 21,3) | … | |

The center of the grid is considered "bull's eye".

Given the grid size and where the dart has landed, determine the score. The picture below illustrates the scoring:

| | | | | | | |
|---|---|---|---|---|---|---|
| 70 | 70 | 70 | 70 | 70 | 70 | 70 |
| 70 | 80 | 80 | 80 | 80 | 80 | 70 |
| 70 | 80 | 90 | 90 | 90 | 80 | 70 |
| 70 | 80 | 90 | **100** | 90 | 80 | 70 |
| 70 | 80 | 90 | 90 | 90 | 80 | 70 |
| 70 | 80 | 80 | 80 | 80 | 80 | 70 |
| 70 | 70 | 70 | 70 | 70 | 70 | 70 |

That is, if the dart is on the bull's eye, score is 100; if the dart is on the square around the bull's eye, score is 90; if the dart is two squares out, score is 80; if the dart is three squares out, score is 70; etc.

## Input

There is only one input line; it contains three integers: $n$ ($3 \le n \le 21$; also $n$ is odd), indicating the grid size; $r$, a valid row number indicating where the dart has landed; and $c$, a valid column number indicating where the dart has landed.

## Output

Print the score.

## Example

| Input | Output |
|---|---|
| 5 2 1 | 80 |
| 13 5 12 | 50 |
| 21 11 11 | 100 |
| 21 1 19 | 0 |
| 21 20 4 | 10 |

```
#include <bits/stdc++.h>
using namespace std;
```

```
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, r, c;
    if (!(cin >> n >> r >> c)) return 0;

    int center = (n + 1) / 2;
    int dist = max(abs(r - center), abs(c - center));
    int score = max(0, 100 - 10 * dist);

    cout << score << "\n";
    return 0;
}
```

# Problem H. Honest (or dishonest) Lottery

| | |
|---|---|
| Source file name: | Honest.c, Honest.cpp, Honest.java, Honest.py |
| Input: | `Standard` |
| Output: | `Standard` |

You suspect your local lottery of cheating! Some numbers are coming up too often!

Each week, the lottery randomly selects five numbers in the range from one to fifty. So, each number should appear about 10% of the time if the numbers are truly chosen randomly. If a number appears far more than that, it's suspicious!

Determine if a set of lottery drawings is suspicious by listing all numbers that appear too often. To allow for random error, you'll need to flag any number that appears more than 20% of the time.

## Input

The first line of input contains a single integer $n$ $\left(1 \le n \le 10^3\right)$. There will be $10 \cdot n$ lottery results for you to analyze.

Each of the next $10 \cdot n$ lines contains 5 integers $x$ $(1 \le x \le 50)$. Each line represents a drawing. All values on a line are unique.

## Output

On a single line, output all numbers that appear strictly more than $2 \cdot n$ times in the list. If there is more than one, output them space-separated, in sorted order from smallest to largest. If there aren't any, output $-1$.

## Example

| Input | Output |
|---|---|
| 1<br>32 30 16 45 27<br>34 45 35 31 42<br>1 12 26 50 13<br>34 50 36 21 39<br>47 7 41 18 45<br>28 48 2 8 4<br>16 40 17 2 19<br>50 4 30 15 6<br>31 13 33 46 18<br>49 23 24 17 48 | 45 50 |

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int h, m;
```

```
    if (!(cin >> h >> m)) return 0;
    cout << (((12 * h) % 360) == m ? "yes" : "no") << "\n";
    return 0;
}
```

Problema J

## Problem J. Jump ... Triple Jump

| | |
|---|---|
| Source file name: | Jump.c, Jump.cpp, Jump.java, Jump.py |
| Input: | Standard |
| Output: | Standard |

You're watching your friend playing a platformer game. In this game, when a player jumps in a straight line, there are three different integer distances they can choose to travel. You don't know what the three are, but hope to find out by watching the gameplay.

You've seen your friend perform many "triple jumps" (i.e. three jumps in a row in the same direction) and recorded the total distance they traveled over all three jumps combined. In each triple jump, your friend can use any of the three integer differences, including repeating the same distance one or more times. After a long time of watching, you've recorded a list of unique distances, and are sure that all possible triple jump distances are in this list.

Given the distances you wrote down, can you recover the three distances?

### Input

The first line of input contains a single integer $n$ ($7 \le n \le 10$), which is the number of unique observations you made.

The next line contains $n$ space separated integers $d$ $\left(1 \le d \le 10^3\right)$. These are your observations, in strictly ascending order.

### Output

Output three space-separated integers on a single line. These are the three jump distances. Output them in sorted order, smallest to largest. The three jump distances are guaranteed to be unique.

### Example

| Input | Output |
|---|---|
| 10<br>6 9 10 12 13 14 15 16 17 18 | 2 5 6 |

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
```

```
    cin >> n;
    vector<int> d(n);
    for (int i = 0; i < n; ++i) cin >> d[i];

    int a = d.front() / 3;
    int c = d.back() / 3;

    for (int mid : d) {
        for (int delta = -3; delta <= 3; ++delta) {
            int b = mid / 3 + delta;
            if (b <= a || b >= c) continue;

            vector<int> vals = {a, b, c};
            vector<int> sums;
            for (int i = 0; i < 3; ++i)
                for (int j = 0; j < 3; ++j)
                    for (int k = 0; k < 3; ++k)
                        sums.push_back(vals[i] + vals[j] + vals[k]);

            sort(sums.begin(), sums.end());
            sums.erase(unique(sums.begin(), sums.end()), sums.end());
            if (sums == d) {
                cout << a << " " << b << " " << c << "\n";
                return 0;
            }
        }
    }
}
```

## Problem K. Knowing the Clock

| | |
|---|---|
| Source file name: | Knowing.c, Knowing.cpp, Knowing.java, Knowing.py |
| Input: | `Standard` |
| Output: | `Standard` |

It is the 25th of November, 1625. You have put a lot of time into making a mechanical watch, but you are unsure if the hands are placed correctly. Even a watch that is not running is correct twice a day, but if the hands of the watch do not correspond to a real time, it is never correct at all. Without wasting any more time, you measure the angles of the hands and check whether they correspond to a real time.

CC BY-SA 4.0 by Jordiferrer on Wikimedia Commons

For example, consider the first sample input, visualized in Figure 4: if the minute hand points to a quarter past (90 degrees from 12 o'clock), then the hour hand cannot point exactly to 2 o'clock (60 degrees from 12 o'clock).

You know that both hands of the watch are moving continuously.

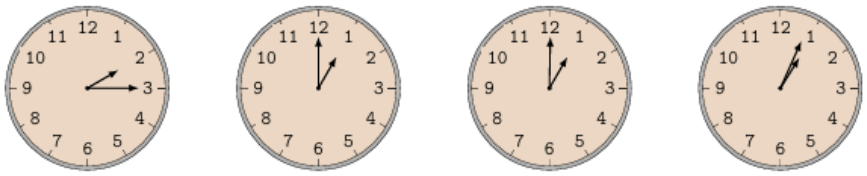

Figure 4: Illustrations of the sample inputs. The first and third sample inputs *do not* correspond to a real time. The second and fourth sample inputs *do* correspond to a real time.

### Input

The input consists of:

- One line with two integers $h$ and $m$ ($0 \le h, m \le 359$), the exact clockwise angle that the hour hand is from 12 o'clock and the exact clockwise angle that the minute hand is from 12 o'clock. Both angles are given in degrees.

### Output

If the angles of the hands correspond to a real time, output "`yes`". Otherwise, output "`no`".

### Example

| Input | Output |
|---|---|
| 60 90 | no |
| 30 0 | yes |
| 30 1 | no |
| 32 24 | yes |

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n, t;
    vector<int> num;
    map<int, int> con;
```

```
    cin >> n;
    int p = 10 * n;

    for (int i = 0; i < p * 5; i++) {
        cin >> t;
        num.push_back(t);
    }

    for (int f : num) {
        con[f]++;
    }

    bool ok = false;
    bool first = true;
    for (auto& par : con) {
        if (par.second > (n * 2)) {
            if (!first) cout << " ";
            cout << par.first;
            first = false;
            ok = true;
        }
    }

    if (!ok) cout << -1;
    cout << endl;

    return 0;
}
```

## Problem K. Koehandel

| | |
|---|---|
| Source file name: | Koehandel.c, Koehandel.cpp, Koehandel.java, Koehandel.py |
| Input: | Standard |
| Output: | Standard |

You are playing a game of *Betting And Purchasing Cows* (BAPC), and Old MacDonald wants to buy one of your cows using a bid-off. This works as follows.

He bets some number of BAPC coins, which he puts in a cup that he places upside down on the table. You then openly bet some number of your own coins. The cup is removed to reveal MacDonald's bet. The bets are exchanged: you receive the bet from MacDonald and he receives your bet. The player that bet the most BAPC coins receives one cow from the opponent, regardless of who started the bidding. In case of a tie, no cows are exchanged.

You love the cows in the game, so your primary goal is to have the most possible cows after the trade. Your secondary goal is to have the highest possible number of coins after the trade. You pay close attention when Old MacDonald is secretly putting their coins in the cup, and by listening very carefully, you have determined the exact number of coins that he put in the cup. Given this information and the number of coins you have, how many coins should you bet?

This cute cow you would not to want to part with for anything.
CC BY-SA 4.0 by Ashish Rajvanshi on Wikimedia Commons

### Input

The input consists of:

- One line with two integers $c$ and $n$ ($0 \le c, n \le 10^9$), the number of BAPC coins in Old MacDonald's cup and the number of coins that you have.

### Output

Output the number of coins that you should bet, keeping your goals in mind.

### Example

| Input | Output |
|---|---|
| 6 19 | 7 |
| 42 37 | 0 |
| 25 25 | 25 |

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    long long oldCoins, myCoins;
    cin >> oldCoins >> myCoins;

    if(myCoins > oldCoins) cout << oldCoins + 1 << endl;
```

```
    else if(myCoins == oldCoins) cout << myCoins << endl;
    else if(myCoins < oldCoins) cout << 0 << endl;


    return 0;
}
```

## Problem B. Bottle of New Port

| | |
|---|---|
| Source file name: | Bottle.c, Bottle.cpp, Bottle.java, Bottle.py |
| Input: | Standard |
| Output: | Standard |

At the Bottles and Port Company, you produce and bottle only the finest of vintages. To satisfy the desires of your highly exclusive clientele, it is necessary to study what happens to your wines once a new bottle is opened. When that happens, liquids in the bottle immediately start evaporating. The alcohol in the bottle evaporates at a different rate than the other liquids in the bottle, meaning that over time, the delicate balance of the port wine will be disturbed.

A bottle of your favourite port, won at a programming competition.
Picture by Reinier Schmiermann

Your goal is to compute how the alcohol percentage changes in the days after opening a fresh bottle. By placing the opened bottle in an advanced chemical apparatus, you are able to measure how fast each liquid in the bottle evaporates.

### Input

The input consists of:

- One line with an integer $d$ ($0 \le d \le 10^6$), the number of days your bottle of new port has remained open.
- One line with two integers $a$ and $o$ ($1 \le a, o \le 10^{12}$), the initial volume of alcohol in the bottle and the initial total volume of other liquids in the bottle, both in µL.
- One line with two integers $\Delta_a$ and $\Delta_o$ ($0 \le \Delta_a, \Delta_o \le 10^{12}$), the evaporation rate of the alcohol and the evaporation rate of the other liquids in the bottle, both in µL/day.

It is guaranteed that the bottle is not empty after leaving it open for $d$ days ($d \cdot \Delta_a < a$ or $d \cdot \Delta_o < o$).

### Output

Output the alcohol percentage after leaving the bottle open for $d$ days.

Your answer should have an absolute or relative error of at most $10^{-6}$.

### Example

| Input | Output |
|---|---|
| 2<br>3 8<br>1 1 | 14.2857142857143 |
| 5<br>11 89<br>2 1 | 1.17647058823529 |
| 10<br>10 40<br>1 1 | 0 |
| 10<br>40 10<br>0 3 | 100 |



```
#include <bits/stdc++.h>
#include <iomanip>
using namespace std;
```

```
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    long double days, alc, oth, chanAlc, chanOth;
    cin >> days >> alc >> oth >> chanAlc >> chanOth;
    long double ap = alc - days * chanAlc, op = oth - days * chanOth;
    ap = max(0.0L, ap);
    op = max(0.0L, op);
    cout << fixed << setprecision(10) << 100.0L * (ap / (ap + op)) << '\n';
    return 0;
}
```

## Problem C. Chess

| | |
|---|---|
| Source file name: | Chess.c, Chess.cpp, Chess.java, Chess.py |
| Input: | `Standard` |
| Output: | `Standard` |

Two chess giants, Vito and Patrik, will play a game of chess this year in front of the iconic theater on Jane Street, finally proving who is the **greatest player of all time**. However, as standard chess has become boring for them, they decided to modify the rules of the game to make it more interesting. We will mention only the rules that are relevant to this task.

The chessboard will be a square matrix with $N$ rows and $N$ columns. Only the chess pieces **knight**, **rook**, and **queen** will be used. The pieces behave in the same way as in standard chess. A rook attacks a square if it is in the same row or column as that rook. A queen also attacks all squares in the same row and column, but in addition to that, it attacks all squares along the same diagonals. Knights attack squares that are two rows and one column away or vice versa. Examples of these moves can be seen in the explanations of the examples.

**Note:** Each piece also attacks the square on which it is placed. Additionally, pieces attack through other pieces, i.e., a piece attacks squares according to the rules stated above regardless of whether there is another piece between the square and the attacking piece.

Vito is preparing for the long-awaited showdown and needs your help. He has decided to practice his quick observation skills. He will do this by placing $M$ chess pieces on the board and then determining all the squares that are attacked. Your task is to determine the number of attacked squares on the given board.

## Input

The first line contains pozitive integers $N$ and $M$ $\left(1 \le N \le 200,\ 1 \le M \le N^2\right)$.

In the next $M$ lines, each line contains a single uppercase letter of the English alphabet, representing the type of piece, which can be one of '`N`', '`R`', and '`Q`', corresponding to knight, rook, and queen, respectively, and integers $r_i$ and $c_i$ $(1 \le r_i, c_i \le N)$, representing the row and column where that piece is located.

At most one piece can be placed on any square of the board.

## Output

In the first and only line, you need to output the number of attacked squares on the board.

## Example

| Input | Output |
|---|---|
| 7 1<br>Q 4 4 | 25 |
| 5 1<br>N 3 3 | 9 |
| 6 3<br>R 1 4<br>Q 2 1<br>N 5 2 | 25 |

```
#include <bits/stdc++.h>
#include <iomanip>
using namespace std;

int n;
vector<vector<char>> seen;

int tiles(char c, int c1, int c2){
    auto mark = [&](int x, int y)->int{
        if (x>=1 && x<=n && y>=1 && y<=n && !seen[x][y]) {
            seen[x][y] = 1;
            return 1;
        }
        return 0;
    };

    int add = 0;
```

```
    add += mark(c1, c2);

    if (c == 'R' || c == 'Q') {
        for (int j = 1; j <= n; ++j) add += mark(c1, j);
        for (int i = 1; i <= n; ++i) add += mark(i, c2);
    }
    if (c == 'Q') {
        for (int k = 1; c1-k >= 1 && c2-k >= 1; ++k) add += mark(c1-k, c2-k);
        for (int k = 1; c1-k >= 1 && c2+k <= n; ++k) add += mark(c1-k, c2+k);
        for (int k = 1; c1+k <= n && c2-k >= 1; ++k) add += mark(c1+k, c2-k);
        for (int k = 1; c1+k <= n && c2+k <= n; ++k) add += mark(c1+k, c2+k);
    }
    if (c == 'N') {
        int d[8][2] = {
            {2,1},{2,-1},{-2,1},{-2,-1},{1,2},{1,-2},{-1,2},{-1,-2}
        };
        for (int i = 0; i < 8; ++i) add += mark(c1 + d[i][0], c2 + d[i][1]);
    }

    return add;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int m, c1, c2, r = 0;
    cin >> n >> m;

    seen.assign(n+1, vector<char>(n+1, 0));

    for (int i = 0; i < m; i++) {
        char c;
        cin >> c >> c1 >> c2;
        r += tiles(c, c1, c2);
    }
```

```
    cout << r << "\n";
    return 0;
}
```

## Problem J. Judge Meetings

| | |
|---|---|
| Source file name: | Judge.c, Judge.cpp, Judge.java, Judge.py |
| Input: | `Standard` |
| Output: | `Standard` |

The judges for the UCF Local Programming Contest have very busy summers. In order to have effective meetings, at least 3 judges have to be present. Since you enjoy Locals and want the contest to go on, given their vacation schedules, help the judges figure out how many days over the summer they could have meetings.

Given the list of days that each judge is away on vacation, determine the number of days that at least 3 judges could meet (are not away on vacation).

### Input

The first input line contains two integers: $n$ ($3 \le n \le 10$), indicating the number of judges, and $m$ ($30 \le m \le 120$), indicating the number of days for summer vacation. These days are numbered from 1 to $m$. Each of the next $n$ input lines will contain information about each judge's vacations. The first item on each of these input lines is an integer, $v$ ($1 \le v \le 10$), indicating the number of vacations that judge has taken. This will be followed by $v$ pairs of integers, $s$ and $e$, indicating that the judge took a vacation starting on day $s$ of the summer and ending on day $e$ of the summer. For all pairs of $s$ and $e$, $1 \le s \le e \le m$. For each judge, no vacations will overlap but it's possible that a judge could end a vacation on one day and leave for another vacation on the following day, meaning that judge is not available for a meeting at all between those two vacations. The vacations for a judge are not necessarily listed in order, i.e., the vacations for a judge are listed in a random order.

### Output

Print the number of days during the summer that at least 3 judges are available to meet (not on vacation).

## Output

Print the number of days during the summer that at least 3 judges are available to meet (not on vacation).

## Example

| Input | Output |
|---|---|
| 5 90<br>2 30 40 50 60<br>1 1 45<br>1 40 90<br>2 33 40 41 50<br>1 45 90 | 62 |
| 3 60<br>2 30 39 1 10<br>1 1 20<br>1 45 60 | 14 |

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n, m;
    cin >> n >> m;

    vector<int> dias(m + 1, 0);

    for (int i = 0; i < n; i++) {
        int v;
        cin >> v;  // número de vacaciones del juez
        for (int j = 0; j < v; j++) {
            int inicio, fin;
            cin >> inicio >> fin;
            for (int d = inicio; d <= fin; d++) {
                dias[d]++;  // ese día el juez está fuera
            }
        }
    }
```

```
    int dias_reunion = 0;
    for (int d = 1; d <= m; d++) {
        int disponibles = n - dias[d];
        if (disponibles >= 3) dias_reunion++;
    }

    cout << dias_reunion << endl;
}
```

## Problem E. Encrypting

| | |
|---|---|
| Source file name: | Encrypting.c, Encrypting.cpp, Encrypting.java, Encrypting.py |
| Input: | `Standard` |
| Output: | `Standard` |

While Mr. Malnar was traveling by bus to Graz, he noticed that other passengers were looking at his phone and reading the messages he was sending to Patrik. Mr. Malnar decided to put an end to this. Therefore, together with Patrik, he developed a new way of encrypting their messages, which they call the *VolksWagen* cipher.

The received message can be imagined as a table of characters with 2 rows and $n$ columns. Each letter spans all 2 rows and several columns, and they are separated by spaces. The appearance of letters in the message can be seen in the example test cases.

The letter '`v`' is represented as follows:

```
\../
.\/.
```

The letter '`w`' is represented as follows:

```
\../\../
.\/..\/.
```

From now on, Patrik and Mr. Malnar will communicate exclusively using the letters '`v`' and '`w`' (without quotes). However, Mr. Malnar is having trouble reading these messages. He has asked you to help him decipher the message he received. Of course, he has not revealed to you how to decrypt their cipher.

## Input

The first line contains a positive integer $N$ ($1 \leq N \leq 1000$), the number of columns in the message.

In the next 2 rows, there are $N$ characters each, representing one row of the message. It is guaranteed that empty columns will be exactly between two different letters, and there will be exactly one empty column between two letters. (*An empty column* is considered to be one that contains only the character '.'.)

## Output

In a single line, print the letters that appear in the message in order.

## Example

| Input | Output |
|---|---|
| 32<br>\../.\../\../.\../.\../.\../\../<br>.\/...\/..\/...\/...\/...\/..\/. | vwvvw |
| 27<br>\../\../.\../\../.\../.\../<br>.\/..\/...\/..\/...\/...\/. | wwvv |

```
#include <bits/stdc++.h>
using namespace std;


int main(){
    ios_base::sync_with_stdio(0);
    cin.tie(0);
    cout.tie(0);

    int n;
    cin >> n;
    cin.ignore(numeric_limits<streamsize>::max(), '\n');

    string lineat;
    string temp;
    string mensaje="";

    getline(cin,lineat);

    cin.ignore(numeric_limits<streamsize>::max(), '\n');
```

```
    getline(cin,temp);

    /*cout << "termino de recibir "<< endl;
    cout << lineat << endl;*/

    int espacio=0;

    for(int i=0;i<n-1;i++){
        //cout << i << " " << lineat[i] << endl;
        if(lineat[i]='.' && lineat[i-1]='/' && lineat[i+1]='\\'){
            if( (i-espacio) == 8 ){
                mensaje+='W';
                //aqui hay una w
                //cout << "W" << " ";
            }else if((i-espacio)==4){
                mensaje+='V';
                //cout << "V" << " ";
            }
            espacio=i+1;
        }

    }

    //cout << "el espacio anteriro " << espacio << endl;
    int i=n-1;
    if((i-espacio)==7){
        mensaje+='W';
        //cout << "W" << " ";
    }else{
        mensaje+='V';
        //cout << "V " << " ";
    }


    cout << mensaje << endl;
    //cout << "hola mundo " << endl;
```

```
    return 0;



}
```

Matriz De Adyacencia

```
#include <iostream>
using namespace std;
int main()
{
    int n,e;
     cin>>n>>e;

     int MATRIZ[n][n];
     for(int i=0; i<n;++i)
         for(int j=0;j<n;++j)
         MATRIZ[i][j]=0;//aqui se limpia la matriz

    for(int i=0;i<e;++i){
        int u,v, w;
        cin>>u>>v>>w;
        MATRIZ[u][v]=w;
        MATRIZ[v][u]=w;

    }
     for(int i=0; i<n;++i){
         for(int j=0;j<n;++j)
         cout<<MATRIZ[i][j]<< " ";
         cout<<"\n";
     }
     return 0;
}
```

DFS Y BFS DINÁMICO

```
#include<bits/stdc++.h>
#include<vector>
using namespace std;

vector<int> adj[8];
vector<int> visi(8,0);
vector<int> ordenamiento;
char ele[8]={'A','B','C','D','E','F','G'};

void dfs(int u) {
if (visi[u] == 1) {
    return;
    }
visi[u] = 1;
//cout<<ele[u]<<" ";
for (int i = 0; i < adj[u].size(); i++) {
int v = adj[u][i];
dfs(v);
    }
    ordenamiento.push_back(u);
}

int main(){
   int n,e,ni;
   cin>>n>>e;


   for(int i=0;i<e;++i){
       int u,v,w;
       cin>>u>>v>>w;
       adj[u].push_back(v);
       //adj[v].push_back(u);
   }
   for(int i=0; i<n;++i){
       cout<<ele[i]<<": ";
```

```
        for(int j=0; j<adj[i].size();++j){
            cout<<ele[adj[i][j]]<< " ";
        }
        cout<<endl;
    }
    //cin>>ni;
//dfs(ni);
for(int i=0; i<n; i++){// aqui puedes cambiar para una nueva orden como  --- f
or(int i=n-1; i>=0;i--){dfs(i)

    dfs(i);
}
for(int i=ordenamiento.size()-1; i>=0; i--){
    cout<<ordenamiento[i]<<" ";
}
return 0;
}
```

CONVEXOS Y NO CONEXOS

```
#include <iostream>
#include <vector>
#include <queue>
#include<stack>
using namespace std;
int main()
{
    int n,e;
    cin>>n>>e;
    int elemento[6]={4,6,9,7,10,11};
    vector<int> adj[n];
    vector<int> visi(n,0);
    for(int i=0;i<e;++i){
        int u,v,w;
```

```
    cin>>u>>v>>w;
    adj[u].push_back(v);
    adj[v].push_back(u);
}

for(int i=0; i<6;++i){
    cout<<i<<": ";
    for(int j=0; j<adj[i].size();++j){
        cout<<adj[i][j]<<" ";
    }
    cout<<endl;
}


int ni;
cin>>ni;

stack<int> q;
q.push(ni);
visi[ni]=1;
cout<<endl<<"DFS: ";
while(!q.empty()){
    int t=q.top();
    cout<<elemento[t]<<" ";
    q.pop();
    for(int i=0; i<adj[t].size();++i){
         int u=adj[t][i];
         if(visi[u]==0){
        q.push(u);
        visi[u]=1;

         }
    }
}
bool conexo=true;
for(int i=0; i<n;++i){
```

```
        if(visi[i]==0){
            conexo=false;
        }
    }
    if(conexo){
        cout<<endl<<"conexo"<<endl;
    }else{
        cout<<endl<<" no conexo"<<endl;
    }

     return 0;
}
```

ABB

```
#include <iostream>
#include <bits/stdc++.h>
using namespace std;

struct nodo {
  int x;
  nodo* izquierdo;
  nodo* derecho;
};

struct nodo* crearCaja(int n) {
  struct nodo *aux = (struct nodo *) malloc (sizeof (struct nodo));
  aux→x = n;
  aux→izquierdo = NULL;
  aux→derecho = NULL;
  return aux;
}

struct nodo* construirArbol(int preorden[], int inorden[], int inicio, int fin, int& i
```

```
ndice) {
  if (inicio > fin)
    return NULL;

  int actual = preorden[indice++];
  nodo* arbol = crearCaja(actual);

  if (inicio == fin)
    return arbol;

  int posicion = -1;
  for (int i = inicio; i <= fin; i++) {
    if (inorden[i] == actual) {
      posicion = i;
      break;
    }
  }

  arbol->izquierdo = construirArbol(preorden, inorden, inicio, posicion - 1, indic
e);
  arbol->derecho = construirArbol(preorden, inorden, posicion + 1, fin, indice);

  return arbol;
}

void postorden(nodo* raiz) {
  if (raiz == NULL)
    return;

  postorden(raiz->izquierdo);
  postorden(raiz->derecho);
  cout << raiz->x << " ";
}

int main() {
  int t;
```

```
  cout << "Ingrese el tamaño de los arreglos: ";
  cin >> t;

  int preorden[t], inorden[t];
  cout << "Ingrese los valores de preorden:" << endl;
  for (int i = 0; i < t; i++) {
    cin >> preorden[i];
  }

  cout << "Ingrese los valores de inorden:" << endl;
  for (int i = 0; i < t; i++) {
    cin >> inorden[i];
  }

  int indice = 0;
  nodo* arbol = construirArbol(preorden, inorden, 0, t - 1, indice);

  cout << "El postorden del arbol construido es: ";
  postorden(arbol);
  cout << endl;

  return 0;
}
```

# Bitwise operators

1. The **& (bitwise AND)** in C takes two numbers as operands and does AND on every bit of two numbers. The result of AND is 1 only if both bits are 1.
2. The **| (bitwise OR)** in C takes two numbers as operands and does OR on every bit of two numbers. The result of OR is 1 if any of the two bits is 1.
3. The **^ (bitwise XOR)** in C takes two numbers as operands and does XOR on every bit of two numbers. The result of XOR is 1 if the two bits are different.

4. The **<< (left shift)** in C takes two numbers, the left shifts the bits of the first operand, and the second operand decides the number of places to shift.
5. The **>> (right shift)** in C takes two numbers, right shifts the bits of the first operand, and the second operand decides the number of places to shift.
6. The **~ (bitwise NOT)** in C takes one number and inverts all bits of it.

| X | Y | X & Y | X \| Y | X ^ Y |
|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 |
| 0 | 1 | 0 | 1 | 1 |
| 1 | 0 | 0 | 1 | 1 |
| 1 | 1 | 1 | 1 | 0 |

```
#include <stdio.h>

int main() {

    // a = 5 (00000101 in 8-bit binary)
    // b = 9 (00001001 in 8-bit binary)
    unsigned int a = 5, b = 9;

    // The result is 00000001
    printf("a&b = %u\n", a & b);
```

```
    // The result is 00001101
    printf("a|b = %u\n", a | b);

    // The result is 00001100
    printf("a^b = %u\n", a ^ b);

    // The result is 11111111111111111111111111111010
    // (assuming 32-bit unsigned int)
    printf("~a = %u\n", a = ~a);

    // The result is 00010010
    printf("b<<1 = %u\n", b << 1);

    // The result is 00000100
    printf("b>>1 = %u\n", b >> 1);
    return 0;
}
```

It is used in many problems. A simple example could be "Given a set of numbers where all elements occur an even number of times except one number, find the odd occurring number" This problem can be efficiently solved by doing XOR to all numbers.

```
#include <stdio.h>

int main(void) {

    int arr[] = { 12, 12, 14, 90, 14, 14, 14 };
    int n = sizeof(arr) / sizeof(arr[0]);
    int res = 0, i;
    for (int i = 0; i < n; i++)
        res ^= arr[i];
    printf("%d", res);
    return 0;
}
```

# Swap bits in a given number

Given a number **x** and two positions (0 based from the right side) in the binary representation of x, write a function that swaps **n** bits at the given two positions and returns the result. It is also given that the two sets of bits do not overlap.

***Input**:*

x = 47 (**001**0**111**1)

p1 = 1 (Start from the second bit from the right side)

p2 = 5 (Start from the 6th bit from the right side)

n = 3 (No of bits to be swapped)

**Output**:

227 (11100011)

**Explanation:**

The 3 bits starting from the second bit (from the right side) are

swapped with 3 bits starting from 6th position (from the right side)

***Input**:*

x = 28 (11100)

p1 = 0 (Start from first bit from right side)

p2 = 3 (Start from 4th bit from right side)

n = 2 (No of bits to be swapped)

**Output**:

7 (00111)

**Explanation:**

The 2 bits starting from 0th position (from right side) are

swapped with 2 bits starting from 4th position (from right side)

steps:

1. Extract the first set of n bits from position p1 by right-shifting and masking with an n-bit mask

2. Extract the second set of n bits from position p2 using the same technique
3. XOR these two sets to identify which corresponding bits differ between them
4. Position these XOR differences back at their original locations (p1 and p2)
5. XOR the original number with these positioned differences to selectively flip only the bits that need to change

```
// C++ Program to swap bits
// in a given number
#include <bits/stdc++.h>
using namespace std;

// Function to swap bits in a given number
int swapBits(int x, int p1, int p2, int n) {

    // Move all bits of first set to rightmost side
    int set1 = (x >> p1) & ((1 << n) - 1);

    // Move all bits of second set to rightmost side
    int set2 = (x >> p2) & ((1 << n) - 1);

    // Xor the two sets
    int xorVal = (set1 ^ set2);

    // Put the Xor bits back to their original positions
    xorVal = (xorVal << p1) | (xorVal << p2);

    // Xor the 'Xor' with the original number so that the
    // two sets are swapped
    int result = x ^ xorVal;

    return result;
}

int main() {
```

```
    int res = swapBits(28, 0, 3, 2);
    cout << res;
    return 0;
}
```